

PLAYWRIGHT FRAMEWORK QUESTIONS

JavaScript • TypeScript • POM • BDD • Playwright Test

Interview Questions & Practical Examples

Framework, Page Object Model, BDD/Cucumber, JavaScript/TypeScript, Playwright Test, CI/CD and real-time scenario questions.

1. Framework & POM Fundamentals

What is an automation framework?

A framework is a structured set of reusable components, conventions and utilities used to build, execute and maintain automated tests.

Which framework have you used and why?

Example: "I have worked with Playwright using JavaScript/TypeScript, POM and BDD. POM separates page locators/actions from test scenarios, while BDD makes business scenarios readable and traceable."

Explain your Playwright framework components.

Typical structure: features, step-definitions, pages, fixtures/hooks, utils, test-data, API helpers, configuration, reports and CI/CD configuration.

What are the benefits of a framework?

Reusable components, centralized locators, faster test development, easier maintenance, clean separation of concerns, data separation, cross-browser execution, parallel execution, retries, tracing/screenshots and CI/CD.

What is POM?

Page Object Model is a design pattern where application pages/components are represented by classes containing locators and reusable actions. Tests focus on business flow rather than selector details.

How many page classes should you create?

Create page/component classes according to application responsibilities and reuse. Avoid one huge page class and avoid unnecessary fragmentation.

Why use a POM object repository?

It centralizes selectors, avoids duplicate locators, improves reusability and makes selector changes easier.

2. POM — JavaScript

How do you store objects in POM?

```
class LoginPage {
  constructor(page) {
    this.page = page;
    this.username = page.getByLabel('Username');
    this.password = page.getByLabel('Password');
    this.loginButton = page.getByRole('button', { name: 'Login' });
  }
  async login(user, pwd) {
    await this.username.fill(user);
    await this.password.fill(pwd);
    await this.loginButton.click();
  }
}
module.exports = { LoginPage };
```

How do you access POM from a test?

```
const { test } = require('@playwright/test');
const { LoginPage } = require('../pages/loginPage');

test('valid login', async ({ page }) => {
  const login = new LoginPage(page);
  await page.goto('/login');
  await login.login('user1', 'password');
});
```

3. POM — TypeScript

Typed POM example

```
import { Page, Locator } from '@playwright/test';

export class LoginPage {
  readonly username: Locator;
  readonly password: Locator;
  readonly loginButton: Locator;

  constructor(private readonly page: Page) {
    this.username = page.getByLabel('Username');
    this.password = page.getByLabel('Password');
    this.loginButton = page.getByRole('button', { name: 'Login' });
  }

  async login(user: string, pwd: string): Promise<void> {
    await this.username.fill(user);
    await this.password.fill(pwd);
    await this.loginButton.click();
  }
}
```

Why TypeScript for automation?

Static typing, better IDE support, safer refactoring, clearer contracts, autocomplete and maintainability in larger frameworks.

4. BDD / Cucumber with Playwright

What is BDD?

Behavior Driven Development expresses expected system behavior in business-readable scenarios. A common format is Given–When–Then using Gherkin.

Why use BDD with Playwright?

BDD can improve communication between QA, developers and business stakeholders by describing behavior in readable scenarios while Playwright performs the browser automation.

What are the main BDD components?

Feature files contain Gherkin scenarios; step definitions map Given/When/Then steps to automation code; page objects contain UI interactions; hooks/fixtures handle setup and teardown; test data supports scenario inputs.

Example feature file

Feature: Login

Scenario: Successful login

```
Given I am on the login page
When I login with valid credentials
Then I should see the dashboard
```

Example step definition — JavaScript

```
Given('I am on the login page', async function () {
  await this.page.goto('/login');
});

When('I login with valid credentials', async function () {
  await this.loginPage.login(this.user, this.password);
});

Then('I should see the dashboard', async function () {
  await expect(this.page).toHaveURL(/dashboard/);
});
```

How do you combine BDD and POM?

Keep Gherkin focused on business behavior, keep step definitions thin, and place locators/UI implementation inside page objects. This prevents step definitions from becoming large automation scripts.

What are hooks in BDD?

Before/After hooks can initialize or clean up test context, browser state, authentication and evidence such as screenshots. Use the framework's fixture/lifecycle capabilities where they provide cleaner dependency management.

What is a good BDD scenario?

A scenario should describe one behavior, use meaningful business language, avoid implementation details such as CSS/XPath, and remain independently executable where practical.

5. Data-Driven Testing

What is data-driven testing?

Running the same scenario against multiple input datasets.

How do you run the same Playwright test with multiple data?

```
const users = [
  { user: 'user1', pwd: 'pwd1' },
  { user: 'user2', pwd: 'pwd2' }
];

for (const data of users) {
  test(`login - ${data.user}`, async ({ page }) => {
    const login = new LoginPage(page);
    await login.login(data.user, data.pwd);
  });
}
```

Where should test data live?

Small datasets can be in JSON/TypeScript objects. Larger datasets can use CSV/Excel/database/API sources. Keep test data separate from page objects.

6. JavaScript / TypeScript OOP in Playwright

Where is encapsulation used?

POM classes encapsulate locators and page actions. Tests call methods such as `login()`, `searchProduct()` or `submitForm()`.

Where is inheritance used?

It can be used selectively with a BasePage containing common helpers. Prefer composition when inheritance does not add clear value.

Where is polymorphism used?

A common interface can represent different implementations of a component or service. Avoid forcing Java-style OOP patterns where they do not simplify the framework.

Where is abstraction used?

Utilities, fixtures, API clients and page methods hide low-level implementation details from test cases.

7. Playwright Test Runner

What is Playwright Test?

Playwright Test provides fixtures, assertions, retries, projects, parallel execution, tracing, screenshots/video options and reporting.

How do you organize setup and teardown?

Use fixtures and beforeEach/afterEach or beforeAll/afterAll where appropriate. Prefer reusable fixtures for common dependencies.

How do you run tests in parallel?

Configure workers/projects in playwright.config.ts or use CLI options. Ensure data and environments are isolated.

How do you execute failed tests?

Use the HTML report, inspect the failure, trace and evidence, then rerun the relevant test. Configure retries where appropriate.

How do you disable a test?

Use test.skip(), test.fixme(), or conditional skip logic.

8. Assertions, Waits & Synchronization

How does Playwright handle synchronization?

Playwright automatically waits for many actionability conditions. Prefer web-first assertions and condition-based waits over fixed sleeps.

How do you handle dynamic elements?

Use resilient locators, locator filters and web-first assertions. Avoid arbitrary waitForTimeout() calls.

How do you wait for an element?

```
await expect(page.getByRole('heading', { name: 'Dashboard' })).toBeVisible();
```

How do you validate UI?

```
await expect(locator).toHaveText('Success');
await expect(locator).toBeVisible();
await expect(page).toHaveURL(/dashboard/);
```

9. Common Playwright Scenarios

How do you handle dropdowns?

```
await page.getByLabel('Country').selectOption('IN');
```

How do you handle auto-suggestions?

Type the partial value, locate the suggestion list, wait for the expected option and select it using a stable locator.

How do you handle a new tab/window?

```
const newPagePromise = context.waitForEvent('page');
await page.getByText('Open report').click();
const newPage = await newPagePromise;
await newPage.waitForLoadState();
```

How do you handle iframes?

```
const frame = page.frameLocator('#payment-frame');
await frame.getByLabel('Card number').fill('4111...');
```

How do you handle file upload?

```
await page.locator('input[type="file"]').setInputFiles('test-data/file.pdf');
```

How do you handle downloads?

```
const downloadPromise = page.waitForEvent('download');
await page.getByText('Download').click();
const download = await downloadPromise;
```

How do you handle browser dialogs?

```
page.on('dialog', async dialog => {
  await dialog.accept();
});
```

10. GitHub & CI/CD

What is the role of GitHub in automation?

Version control, collaboration, pull requests, code review, branch management and CI workflows.

How do you integrate Playwright into CI/CD?

Typical flow: checkout → install Node dependencies → install Playwright browsers → run tests → publish report/artifacts → notify results.

What should be stored as CI artifacts?

HTML reports, traces, screenshots/videos when enabled, logs and relevant test output.

11. Playwright Configuration

What is playwright.config.ts?

The central configuration file for test directory, projects, browser settings, baseURL, retries, workers, reporters, timeouts and other test options.

Example configuration

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  testDir: './tests',
  use: { baseURL: 'https://example.com' },
  retries: 1,
  workers: 2,
  reporter: [['html', { open: 'never' }]],
});
```

12. Real-Time Interview Scenarios

100 tests ran and a few failed. What is your first approach?

Check the report and failure details. Determine whether failures are product defects, locator/synchronization issues, environment/data issues or framework problems. Inspect trace, screenshots and logs before rerunning.

All tests failed. What could be wrong?

Check environment availability, base URL, authentication/session state, browser installation, test data, network/service dependencies and recent configuration/framework changes.

When do you automate a test?

Prioritize stable, repeatable and high-value regression scenarios. Consider frequency, risk, maintenance cost and automation feasibility.

What is not suitable for automation?

Some subjective visual checks, one-off exploratory work, unstable requirements, CAPTCHA restrictions or cases where setup and maintenance cost outweigh value.

What is your role in the framework?

Example: developing page objects, reusable business methods, fixtures/utilities, API helpers, test data handling, reporting, CI/CD integration and framework improvements.

13. Quick Revision Checklist

Framework

POM • BDD • Cucumber • fixtures • utilities • config • test data • reporting • CI/CD

JavaScript

let/const • closures • promises • async/await • arrays • destructuring • spread/rest • event loop

TypeScript

types • interfaces • access modifiers • generics • typed page objects • strict mode

Playwright

locators • assertions • auto-waiting • fixtures • projects • parallelism • retries • trace • API testing

BDD

Gherkin • Feature • Scenario • Given/When/Then • Step Definitions • Hooks • POM integration

Advanced

frames • popups • new pages • downloads/uploads • network interception • API mocking • storageState • authentication